Implementing Automated CI/CD Pipelines for ACEest Fitness & Gym

Introduction to DevOps | Assignment I

Abstract

This project demonstrates the implementation of a modern DevOps CI/CD pipeline for a Flask-based Gym Management application. The objective is to automate the development lifecycle from local development to automated build validation and containerized deployment

Assignment Submission Date: 03rd April 2026

BOLLAPRAGADA NAGA RATNA KUMAR | 2025HT66015

2025HT66015@wilp.bits-pilani.ac.in

Table of Contents

Assignment Overview: 2
Assignment: ACEest Fitness CI/CD Pipeline Implementation 4
1. Project Introduction 4
2. CI/CD Pipeline Configuration 4
2.1 Jenkins Automation 4
2.2 GitHub Actions Integration 7
3. Build & Deployment Validation 8
3.1 Jenkins Build Success 8
3.2 Docker Containerization 8
4. Application Functionality & User Roles 10
4.1 Authentication & Registration 10
4.2 Admin (Gym Manager) Dashboard 10
4.3 Trainer & Member Dashboards 12
5. Testing & API Validation 13
5.1 Automated Pytest Results 13
5.2 Postman API Testing 13
6. Conclusion 14

Figure Index

Figure 1: My GitHub Profile 4
Figure 2: BITS VM Jenkins Build Success Console – Screenshot 1 5
Figure 3: BITS VM Jenkins Build Success Console – Screenshot 2 6
Figure 4: BITS VM Jenkins Build Success Console – Screenshot 3 (Status) 6
Figure 5: GitHub Actions – Success (Screenshot) 7
Figure 6: Local Docker Desktop – Containers 8
Figure 7: BITS VM Docker Console – Container Images 8
Figure 8: BITS VM Docker Console – Container Status 9
Figure 9: BITS VM (127.0.0.1) – App running in Docker – Login Page 10
Figure 10: BITS VM (127.0.0.1) – App running in Docker – Admin Dashboard 11
Figure 11: BITS VM (127.0.0.1) – App running in Docker – Trainer Assignment 11
Figure 12: BITS VM (127.0.0.1) – App running in Docker – Trainer Dashboard 12
Figure 13: BITS VM (127.0.0.1) – App running in Docker – Member Dashboard 12
Figure 14: BITS VM (127.0.0.1) – pytest output console 13
Figure 15: Local Postman – HEALTH CHECK STATUS 13
Figure 16: Local Postman – POST MEMBERS STATUS 14

Course Introduction to DEVOPS (Merged - CSIZG514/SEZG514/SEUSZG514)(S2-25)

Assignment - 1

Assignment Overview: **Implementing Automated CI/CD Pipelines for ACEest Fitness & Gym**

Objective - This assignment is designed to provide students with comprehensive, hands-on experience in modern DevOps methodologies. By executing this project, students will attain professional proficiency in **Version Control** (Git/GitHub), **Containerization** (Docker), and the orchestration of **Continuous Integration and Continuous Delivery (CI/CD)** pipelines using **GitHub Actions** and **Jenkins**.

Problem Statement- You have been appointed as a **Junior DevOps Engineer** for **ACEest Fitness & Gym**, a rapidly scaling startup. Your mission is to architect and implement a robust, automated deployment workflow that guarantees code integrity, environmental consistency, and rapid delivery.

Your solution must transition the application through a rigorous lifecycle—from local development to an automated **Jenkins BUILD** environment.

Core Assignment Phases

1. **Application Development & Modularization** Develop a foundational **Flask** web application tailored for fitness and gym management. You will be provided with a baseline Python script to initialize the core logic and service endpoints.

2. **Version Control System (VCS) Strategy** Initialize a local **Git** repository and synchronize it with a remote **GitHub** counterpart. You are expected to follow industry standards for versioning, including descriptive commit messages and branch management to track features, bug fixes, and infrastructure updates.

3. **Unit Testing & Validation Framework** Integrate the **Pytest** framework to develop a comprehensive suite of unit tests. These tests must validate the internal logic of the Flask application, ensuring that all components perform according to specification before they reach the build stage.

4. **Containerization with Docker** Encapsulate the Flask application, along with its environment and dependencies, into a portable **Docker Image**. This ensures "write once, run anywhere" consistency, eliminating the "it works on my machine" syndrome during the transition from testing to production.

5. **The Jenkins BUILD & Quality Gate** Integrate a **Jenkins** server to handle the primary **BUILD** phase. You must configure a Jenkins project that pulls the latest code from GitHub and performs a clean build of the environment. This serves as a secondary validation layer to ensure the code compiles and integrates correctly in a controlled build environment.
6. **Automated CI/CD Pipeline via GitHub Actions** Design a fully automated pipeline using **GitHub Actions** (.github/workflows/main.yml). The pipeline must be triggered by every push or pull_request, executing the following critical stages:
    - **Build & Lint:** Compile the application and check for syntax errors.
    - **Docker Image Assembly:** Successfully build the Docker container.
    - **Automated Testing:** Execute the **Pytest** suite inside the containerized environment to confirm stability.

Required Deliverables

Students must submit a link to a **publicly accessible GitHub Repository** containing:

- **Source Code:** All Flask application files (app.py, requirements.txt).
- **Test Suite:** All Pytest script files.
- **Infrastructure as Code:** A functional Dockerfile and the GitHub Actions YAML configuration.
- **Documentation:** A professional README.md detailing:
    - Local setup and execution instructions.
    - Steps to run tests manually.
    - A high-level overview of the Jenkins and GitHub Actions integration logic.

Evaluation Criteria

Submissions will be assessed based on the following professional benchmarks:

- **Application Integrity:** Does the Flask application function as specified.
- **VCS Maturity:** Are Git commits meaningful and logically structured.
- **Testing Coverage:** Do the Pytest cases effectively cover core functionalities.
- **Docker Efficiency:** Is the Dockerfile optimized for size and security.
- **Pipeline Reliability:** * Does the **Jenkins BUILD** trigger and complete without errors.
    - Does the GitHub Actions workflow successfully pass the Build and Test stages on every push.
- **Documentation Clarity:** Is the README.md professional and easy to follow for other engineers.

Assignment: ACEest Fitness CI/CD Pipeline Implementation

Student Name: BOLLAPRAGADA NAGA RATNA KUMAR

Student ID: 2025HT66015

1. Project Introduction

This project demonstrates the implementation of a modern DevOps CI/CD pipeline for a Flask-based Gym Management application. The objective is to automate the development lifecycle from local development to automated build validation and containerized deployment.

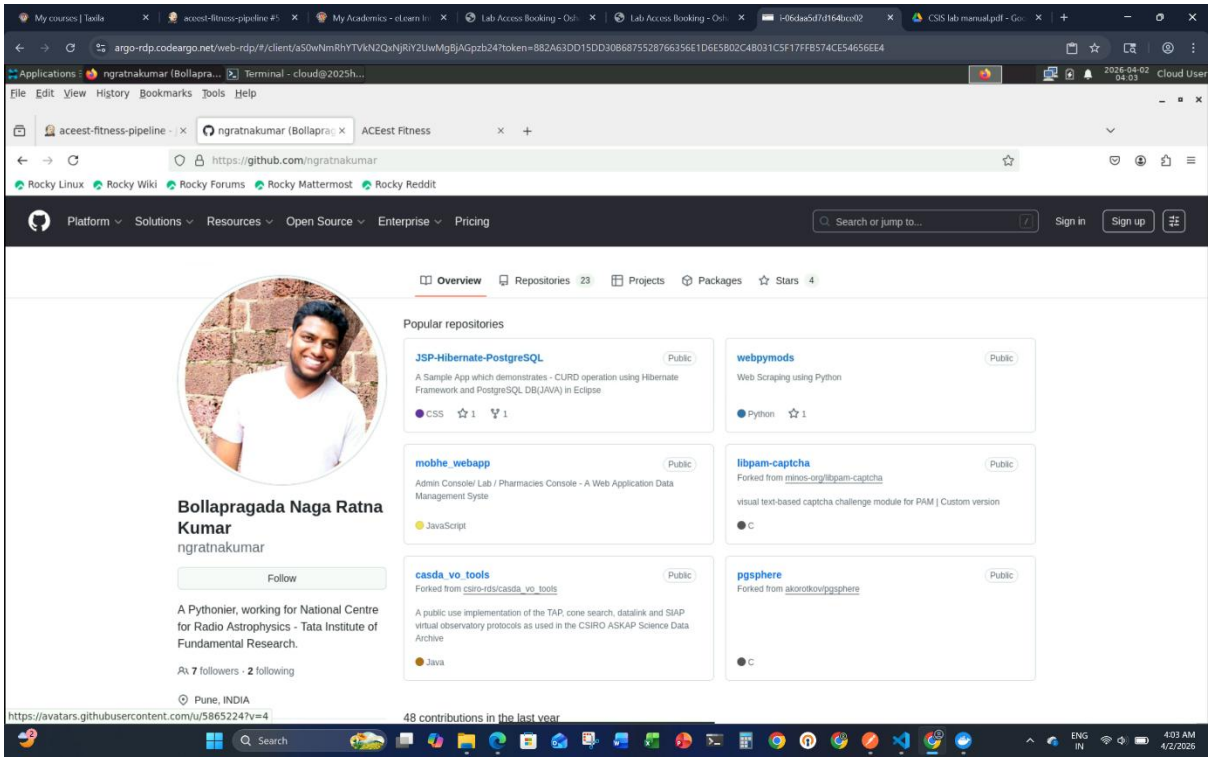
Application Overview: A management system that allows for user registration, trainer assignment, and workout plan management through a web interface and REST API endpoints.

2. CI/CD Pipeline Configuration

2.1 Jenkins Automation

The Jenkins pipeline is configured to automatically pull the latest code from the GitHub repository: **https://github.com/ngratnakumar/aceest-fitness-cicd-pipeline.git**.

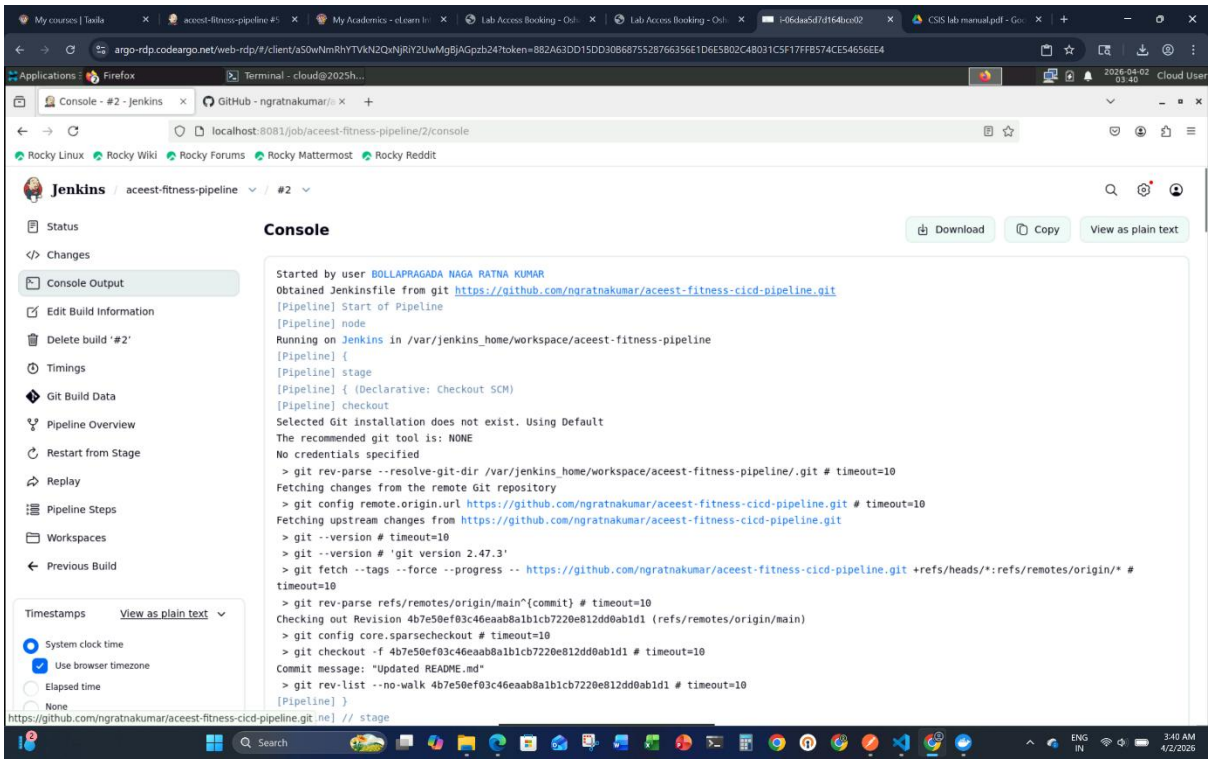
Figure 1: My GitHub Profile



Build Steps: The build process is automated via an "Execute Shell" block consisting of the following tasks:

1. Environment: Creation and activation of a Python virtual environment (venv).
2. Dependencies: Installing required packages via pip install -r requirements.txt.
3. Testing: Running pytest to ensure code stability.
4. Containerization: Building the Docker image (aceest-fit-app) and validating the local image list.
5. Deployment: Running the application container on port 5000:5000.Jenkins Configuration & Steps:

Figure 2: BITS VM Jenkins Build Success Console – Screenshot 1



GitHub Actions Success:

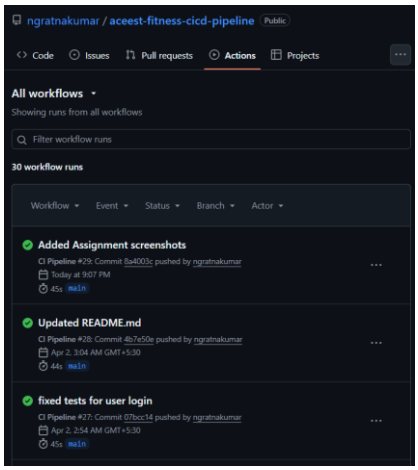


Figure 3: BITS VM Jenkins Build Success Console – Screenshot 2

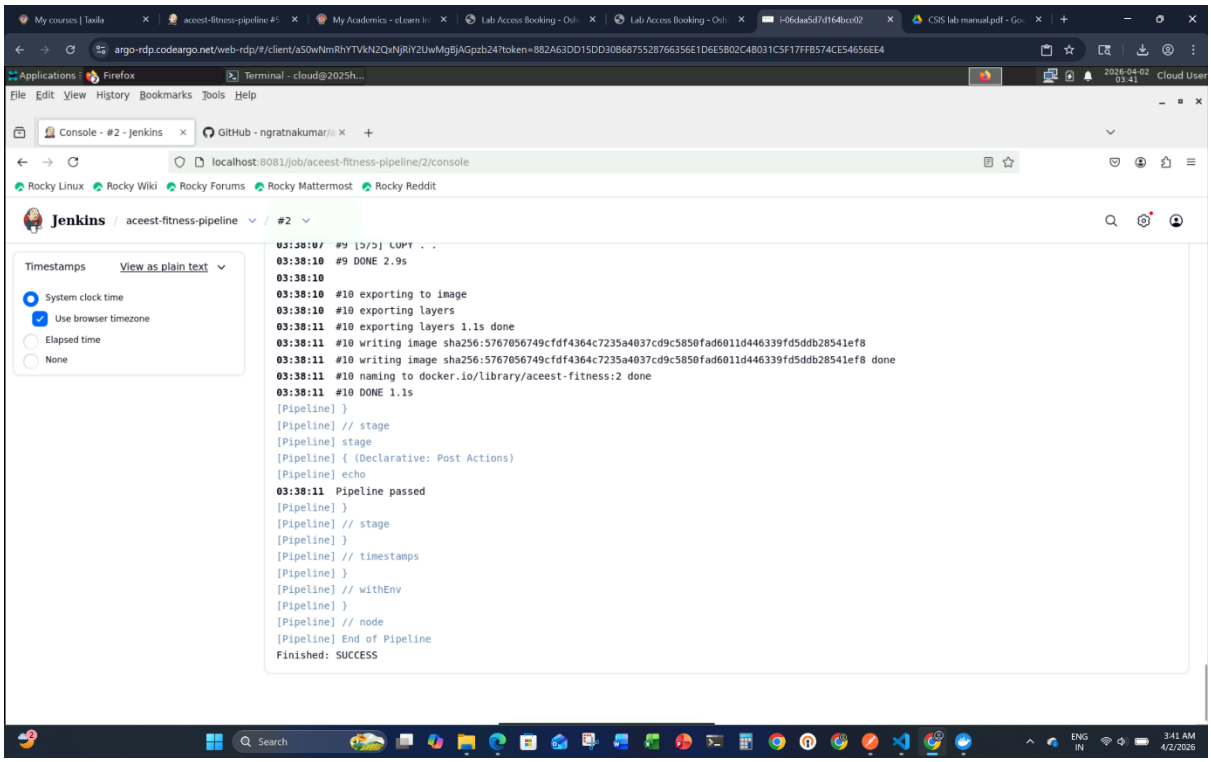
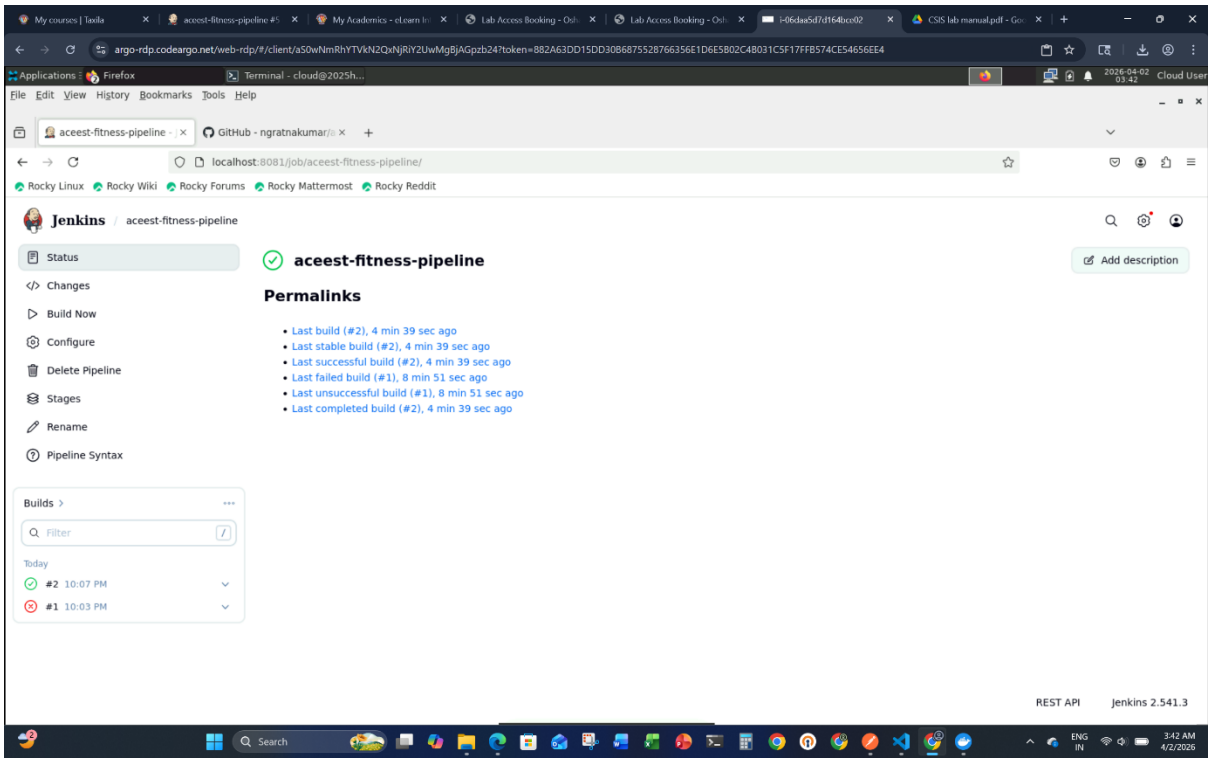


Figure 4: BITS VM Jenkins Build Success Console – Screenshot 3 (Status)

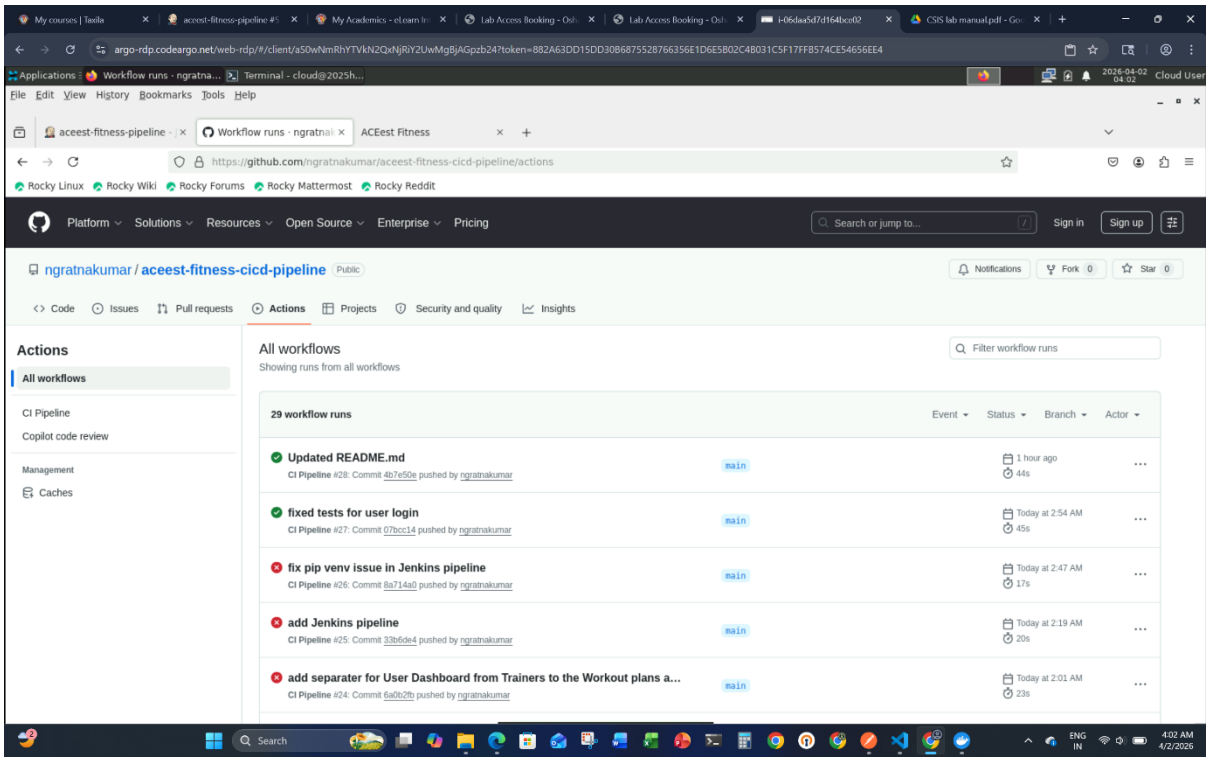


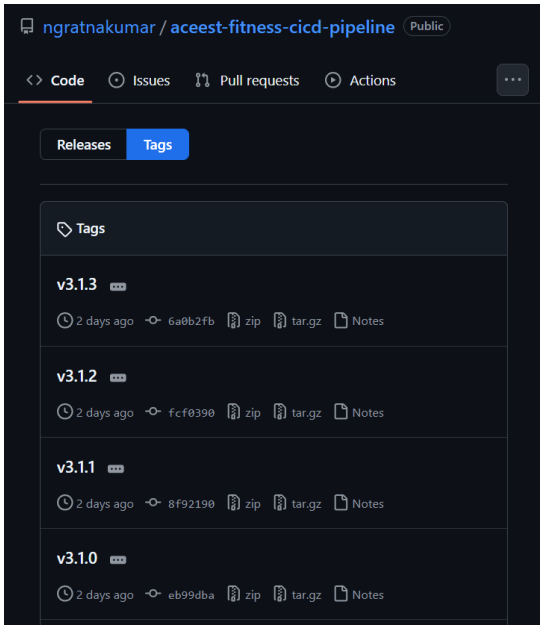
2.2 GitHub Actions Integration

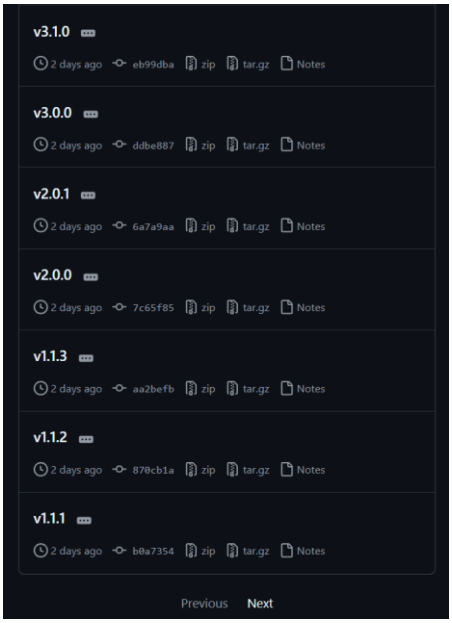
In addition to Jenkins, GitHub Actions provides a secondary CI layer, running workflows for every push to the main branch. This includes setting up Python, installing dependencies, and running tests (e.g., 4 tests passed in 0.16s).

GitHub Actions Success Log:

Figure 5: GitHub Actions – Success (Screenshot)







3. Build & Deployment Validation

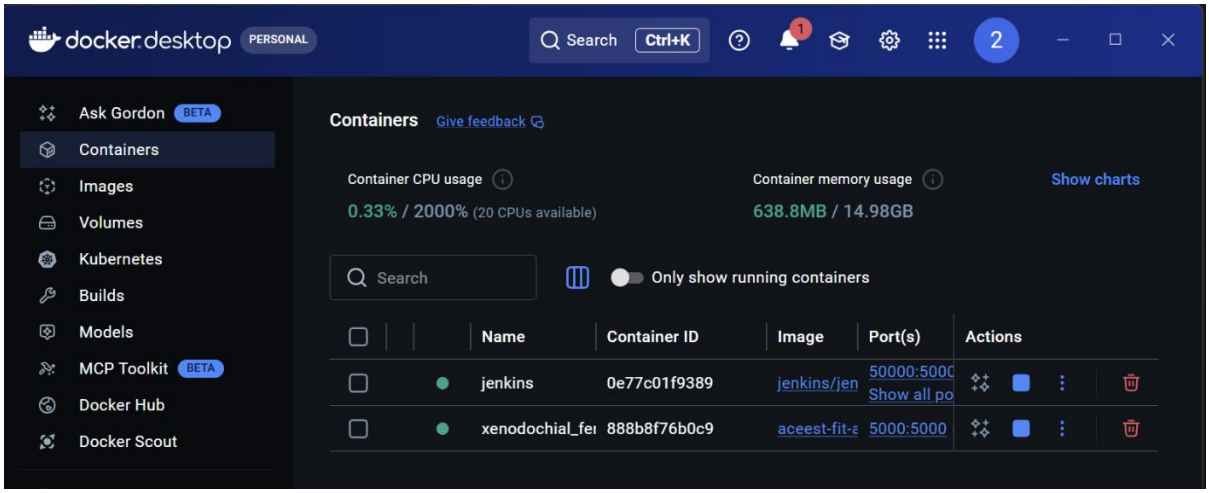
3.1 Jenkins Build Success

The Jenkins dashboard confirms a successful automation cycle. For instance, **Build #2** and **Build #4** are marked as stable, with the console output showing a **"Finished: SUCCESS"** status.

3.2 Docker Containerization

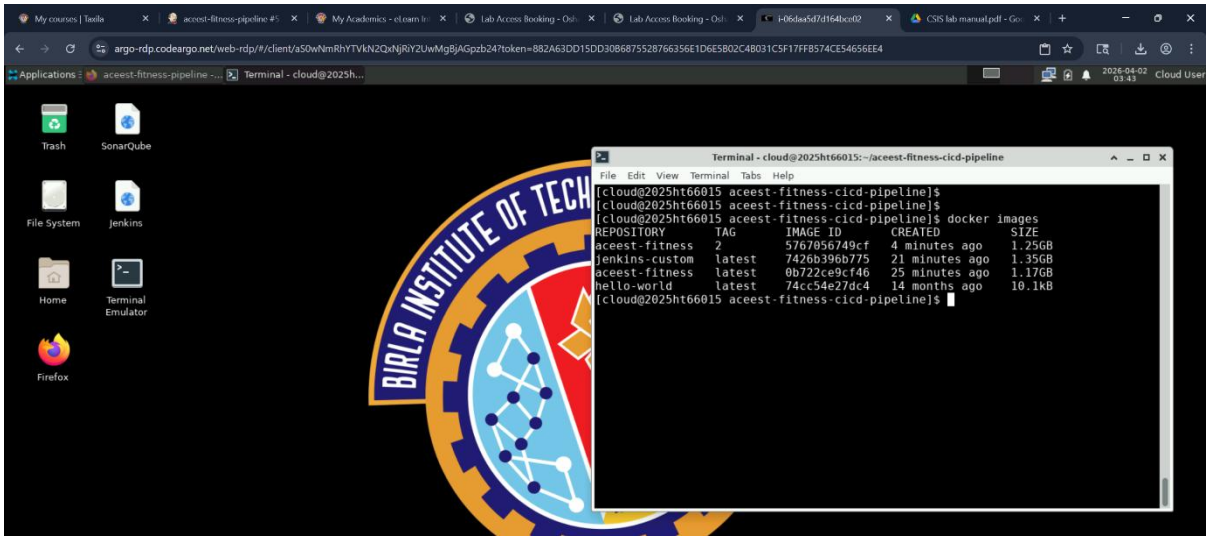
The application is successfully containerized. Both the Jenkins server and the application (aceest-app) are shown running in the Docker environment.

Figure 6: Local Docker Desktop – Containers



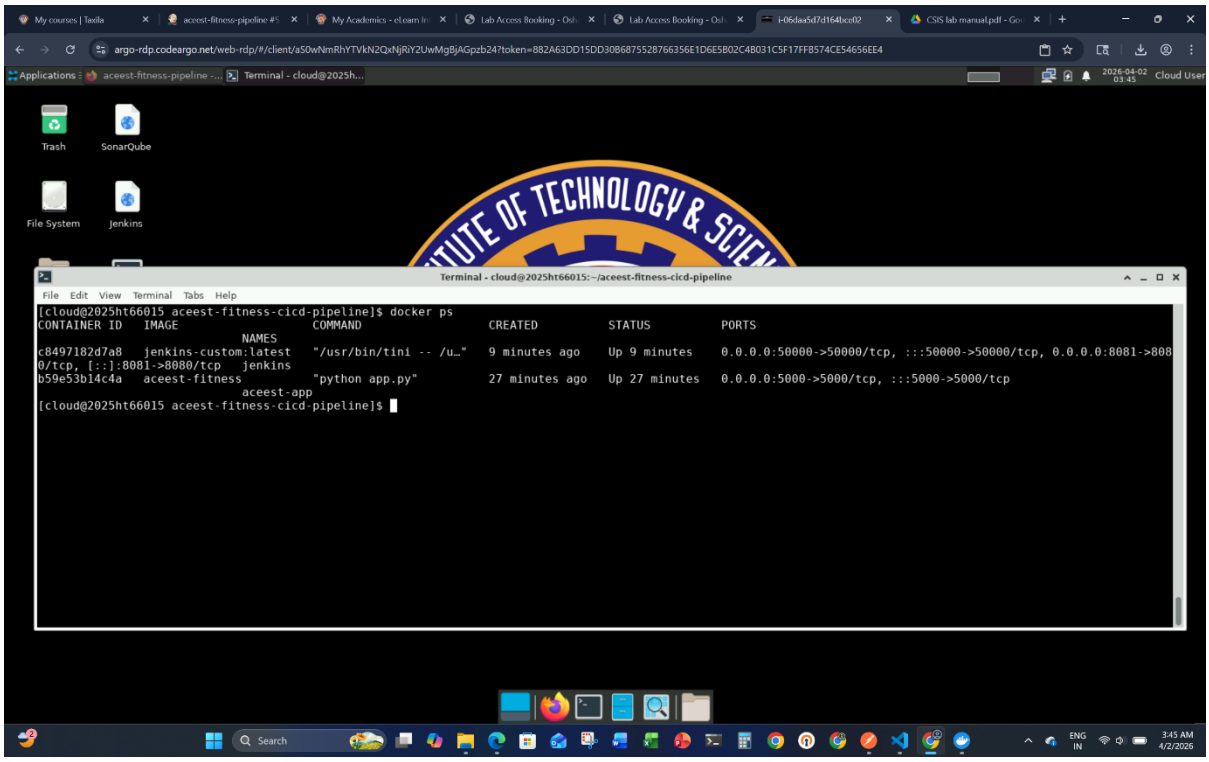
- *Docker Images:* The image aceest-fitness (Tag 2) is created with a size of 1.25GB.

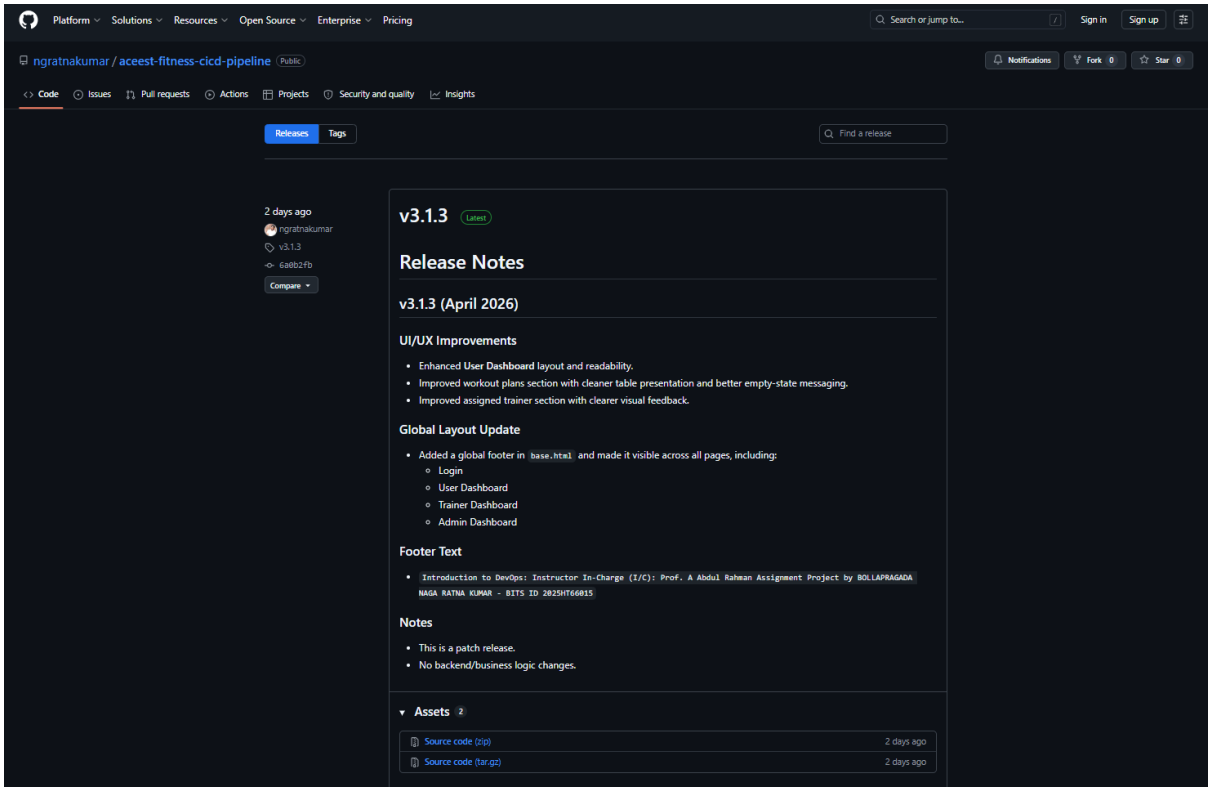
Figure 7: BITS VM Docker Console – Container Images



- *Docker Status:* Deployment is verified through the Docker Desktop UI and terminal docker ps commands.

Figure 8: BITS VM Docker Console – Container Status



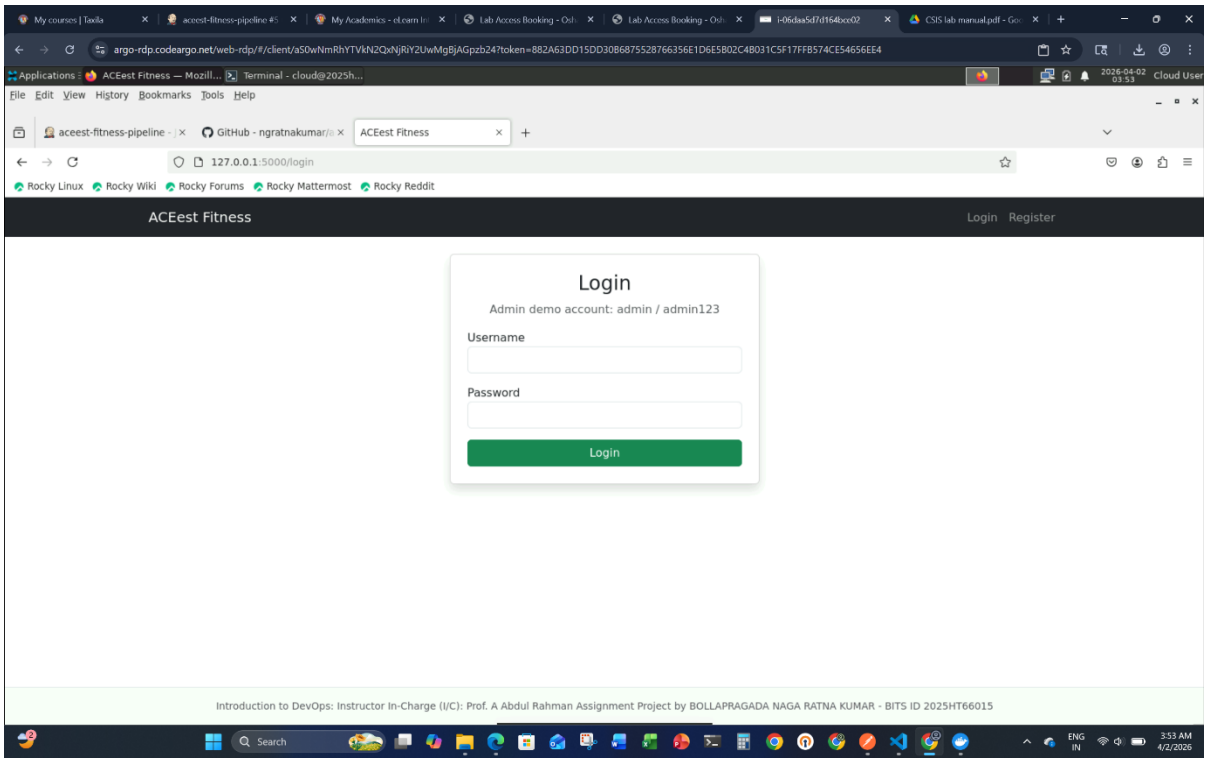


4. Application Functionality & User Roles

4.1 Authentication & Registration

Users can register for specific roles—**Member**, **Trainer**, or **Gym Manager (Admin)**—using the registration portal. A pre-configured admin account (admin / admin123) is available for initial system setup.

Figure 9: BITS VM (127.0.0.1) – App running in Docker – Login Page



4.2 Admin (Gym Manager) Dashboard

The Admin has the authority to add new trainers with specific specializations and assign them to registered members.

- *Assignment Process:* As seen in the dashboard, the user "ratnakumar" was successfully assigned to "new_trainer".

Figure 10: BITS VM (127.0.0.1) – App running in Docker – Admin Dashboard

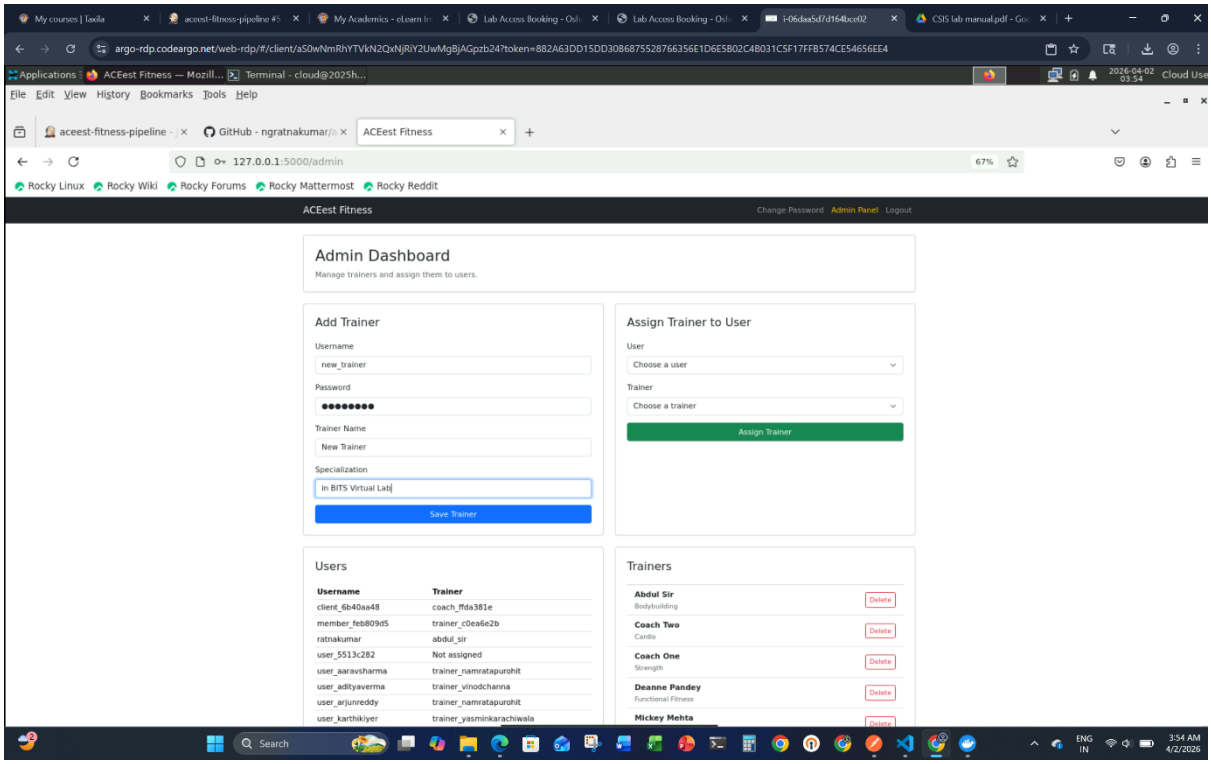
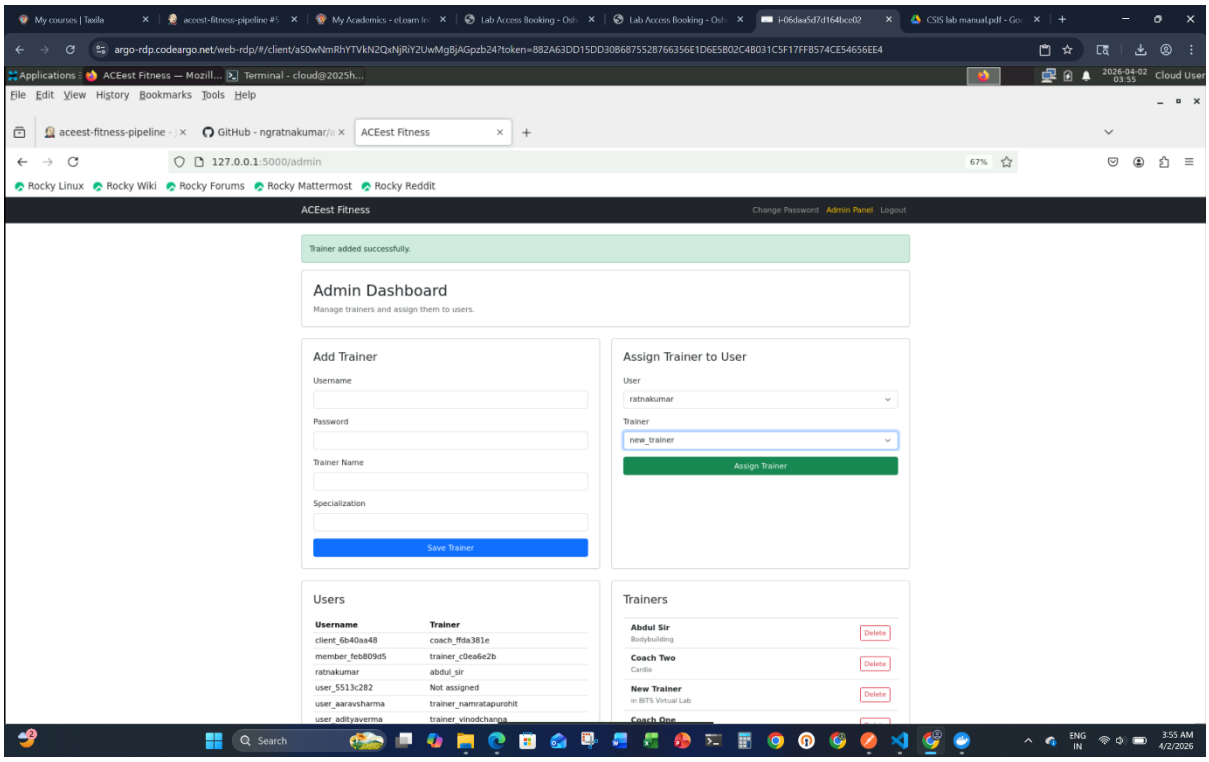


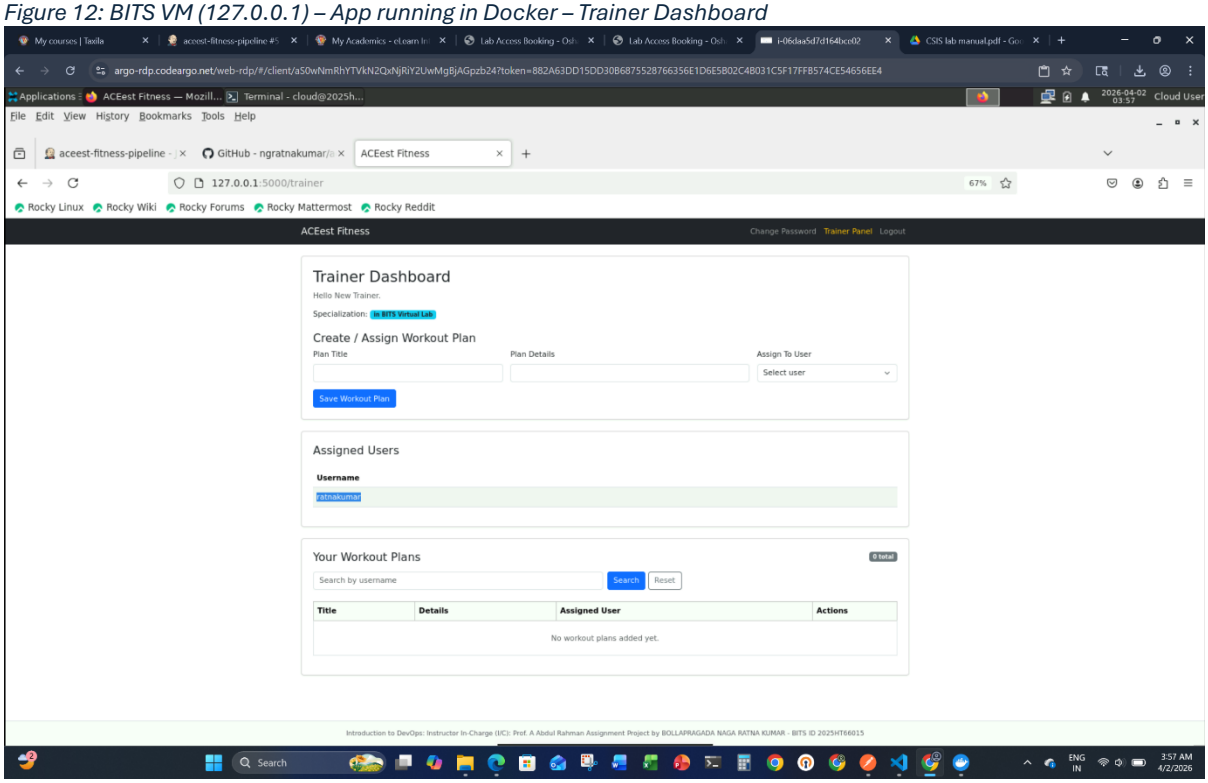
Figure 11: BITS VM (127.0.0.1) – App running in Docker – Trainer Assignment



4.3 Trainer & Member Dashboards

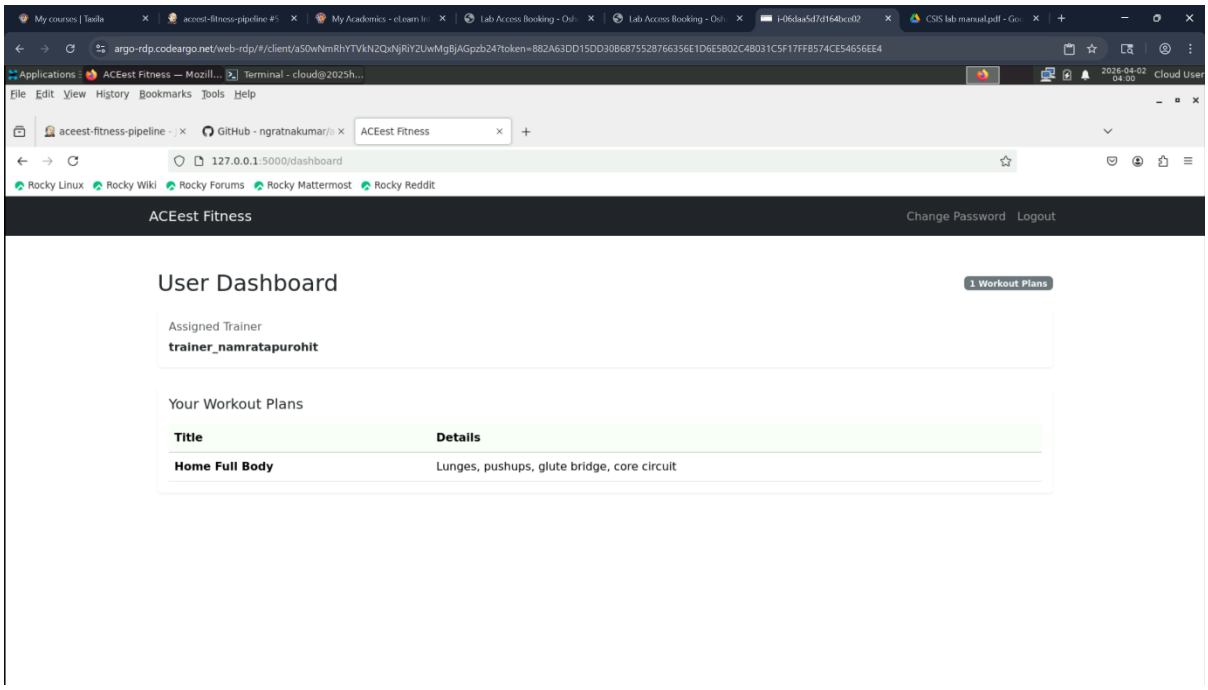
- *Trainer:* After logging in, trainers see their assigned members. They can create workout plans (e.g., "Plan 1" for Planks) which are then saved to the system.

Figure 12: BITS VM (127.0.0.1) – App running in Docker – Trainer Dashboard



- *Member:* Users can log in to view their assigned trainer and their specific workout plan details (e.g., "Home Full Body" workout).

Figure 13: BITS VM (127.0.0.1) – App running in Docker – Member Dashboard

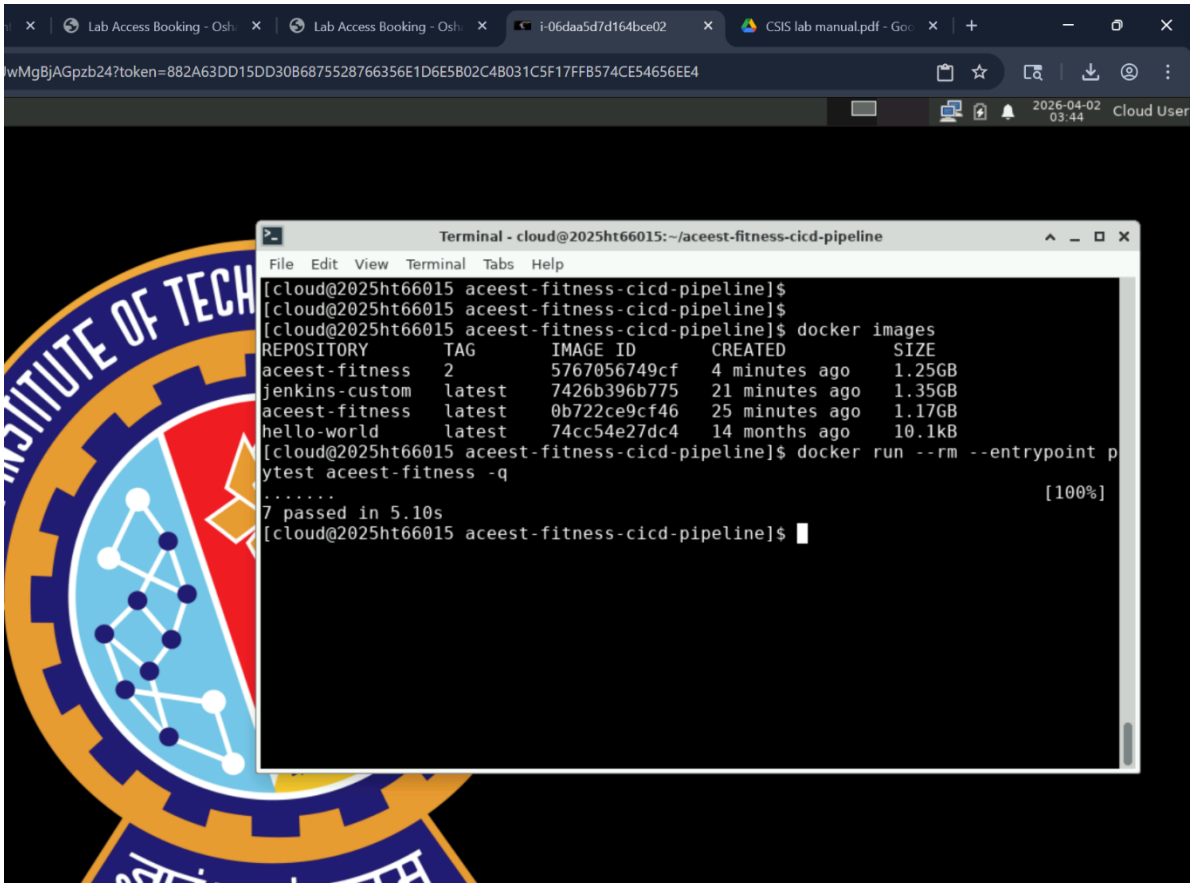


5. Testing & API Validation

5.1 Automated Pytest Results

The system undergoes automated testing both locally and in the CI environment. Local terminal execution confirms that **7 tests passed** in 5.10 seconds.

Figure 14: BITS VM (127.0.0.1) – pytest output console

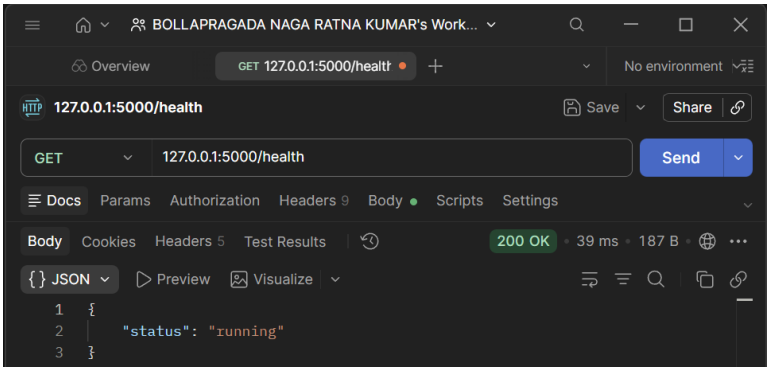


5.2 Postman API Testing

The application's REST API is validated using Postman:

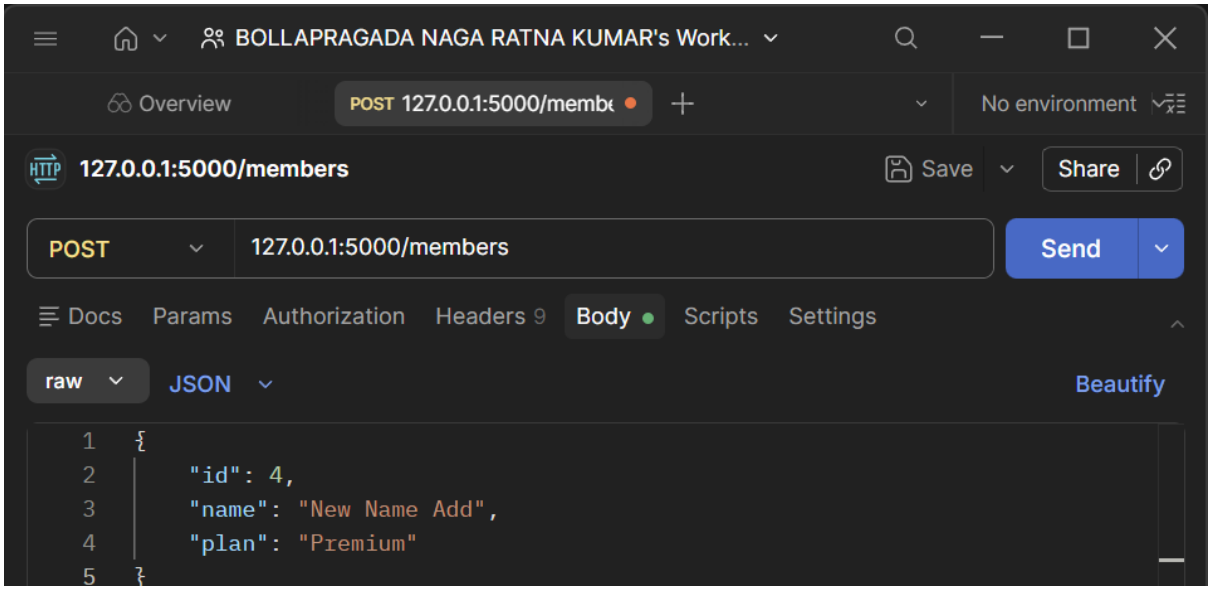
- **Health Check:** GET /health confirms the status is "running".

Figure 15: Local Postman – HEALTH CHECK STATUS



- **Member Management:** Testing includes retrieving the member list and successfully adding a new member via a POST request (id: 4, name: "New Name Add").

Figure 16: Local Postman – POST MEMBERS STATUS



6. Conclusion

The **ACEest Fitness & Gym Management System** successfully demonstrates a professional CI/CD implementation. By combining **Jenkins**, **GitHub Actions**, **Pytest**, and **Docker**, the project ensures that code changes are automatically tested, built, and deployed into a containerized environment, maintaining high application reliability and role-based access control.

BOLLAPRAGADA NAGA RATNA KUMAR

BITS ID: 2025HT66015